

DappToken ICO – Smart Contract Suite

Version: 1.0

Date: June 2026

Author: Fadi Tayih

A fully tested, feature-rich Initial Coin Offering (ICO) smart contract suite built with Solidity 0.4.26, OpenZeppelin 1.12.0, and Truffle 5.11.5. The project includes a mintable/pausable ERC20 token, a multi-stage crowdsale with whitelisting, caps, refunds, and token vesting, and a complete test suite (27 passing tests).

Table of Contents

1. [Key Features](#)
2. [Technology Stack](#)
3. [Smart Contracts](#)
 - [DappToken.sol](#)
 - [DappTokenCrowdsale.sol](#)
4. [Test Suite](#)
5. [Deployment Guide](#)
6. [Usage](#)
7. [Security Considerations](#)
8. [License](#)

1. Key Features

- **Mintable & Pausable ERC20 Token** – DappToken (DAPP) with owner-controlled minting and pausing.
- **Multi-stage Crowdsale** – Pre-ICO and ICO stages with adjustable rates (500 / 250).
- **Hard-capped & Refundable** – If the funding goal is not met, investors can claim refunds via a RefundVault.
- **Whitelisted** – Only approved addresses can participate; ownership-controlled whitelist management.
- **Timed Crowdsale** – Automatic opening and closing times, with time-travel support in tests.
- **Investor Caps** – Minimum (0.002 ETH) and maximum (50 ETH) contribution per address.
- **Token Vesting** – Founders/partners tokens locked in `TokenTimelock` contracts and released after a set period.
- **Pausable Transfers** – Token transfers blocked during crowdsale; token unpaused upon successful finalization.
- **Complete Test Suite** – 27 tests covering minting, caps, stage transitions, refunds, vesting, and admin controls.

2. Technology Stack

Component	Version / Description
Smart Contract Language	Solidity 0.4.26
Framework	Truffle 5.11.5
Node.js	24.16.0
Testing	Chai, Chai-as-promised
OpenZeppelin Library	1.12.0 (battle-tested contracts)
Local Blockchain	Ganache-cli

Deployment Network	Sepolia testnet (or any Ethereum-compatible network)
--------------------	--

3. Smart Contracts

3.1 DappToken.sol

An ERC20 token with pausing, minting, and detailed metadata. It inherits from OpenZeppelin's `StandardToken`, `DetailedERC20`, `PausableToken`, and `MintableToken`.

```
pragma solidity ^0.4.24;

import "openzeppelin-solidity/contracts/token/ERC20/StandardToken.sol"
import "openzeppelin-solidity/contracts/token/ERC20/DetailedERC20.sol"
import "openzeppelin-solidity/contracts/token/ERC20/PausableToken.sol"
import "openzeppelin-solidity/contracts/token/ERC20/MintableToken.sol"

contract DappToken is StandardToken, DetailedERC20, PausableToken, Min
    constructor(string _name, string _symbol, uint _decimals)
        DetailedERC20(_name, _symbol, uint8(_decimals))
    public
    {}
}
```

Roles: Owner can pause/unpause, mint, and finish minting. Transfers are blocked while paused.

3.2 DappTokenCrowdsale.sol

This is the main crowdsale contract. It combines several OpenZeppelin crowdsale modules and adds investor caps, a two-stage ICO, and token vesting logic.

```

contract DappTokenCrowdsale is
    Crowdsale,
    MintedCrowdsale,
    CappedCrowdsale,
    TimedCrowdsale,
    WhitelistedCrowdsale,
    RefundableCrowdsale
{
    // Investor caps
    uint256 public investorMinCap = 20000000000000000; // 0.002 ether
    uint256 public investorHardCap = 5000000000000000000; // 50 ether
    mapping(address => uint256) public contributions;

    enum CrowdsaleStage { PreICO, ICO }
    CrowdsaleStage public stage = CrowdsaleStage.PreICO;

    // Token distribution percentages
    uint256 public tokenSalePercentage = 70;
    uint256 public foundersPercentage = 10;
    uint256 public foundationPercentage = 10;
    uint256 public partnersPercentage = 10;

    address public foundersFund;
    address public foundationFund;
    address public partnersFund;
    uint256 public releaseTime;
    address public foundersTimelock;
    address public foundationTimelock;
    address public partnersTimelock;

    constructor( ... ) public Crowdsale(...) CappedCrowdsale(...) Time

    function setCrowdsaleStage(uint _stage) public onlyOwner { ... }
    function _forwardFunds() internal { ... }
    function _preValidatePurchase(address _beneficiary, uint256 _weiAm
    function finalization() internal { ... }
}

```

Key mechanics:

- Stage-dependent forwarding: during Pre-ICO, funds go to the wallet; during ICO, they go to the refund vault.
- Owner can switch stages and rates (Pre-ICO: 500, ICO: 250).
- Investor contributions are tracked and validated against min/max caps.
- On successful finalization, bonus tokens are minted to founders/foundation/partners via `TokenTimelock` contracts.
- The token is unpaused and ownership transferred back to the wallet.

4. Test Suite

The test suite is located in `test/DappTokenCrowdsale.test.js` and `test/DappToken.test.js`. It uses a custom time-travel helper and manual ABIs for missing artifacts.

Test Coverage (27 tests):

Category	Tests
Basic token properties	Name, symbol, decimals
Crowdsale parameters	Rate, wallet, token address, cap, open/closed state
Minting	Token supply increases after purchase
Whitelist	Rejects non-whitelisted buyers
Refunds	Prevents refund during crowdsale; allows refund after unsuccessful finalization
Fund forwarding	Pre-ICO → wallet, ICO → refund vault

Stage management	Starts in Pre-ICO; admin can switch stages & rates; non-admin cannot
Payment acceptance	sendTransaction and buyTokens work
Investor caps	Rejects below min, allows above after first purchase, rejects above hard cap
Token transfers	Blocked during crowdsale (paused)
Finalization (goal not reached)	Investor can claim refund
Finalization (goal reached)	Minting finishes, token unpauses, vesting balances correct, timelocks prevent early withdrawal, release works after time, ownership transferred, refund blocked
Token distribution	Percentage getters return correct values; total = 100%

All tests pass successfully, validating the contract's correctness.

5. Deployment Guide

Local Development

```
# Start Ganache in a separate terminal
ganache-cli

# Compile contracts
truffle compile
```

```
# Run tests
truffle test

# Deploy to local network
truffle migrate --reset
```

Sepolia Testnet

1. Install dependencies:

```
npm install --save-dev @truffle/hdwallet-provider dotenv
```

2. Create a `.env` file:

```
MNEMONIC="your twelve word mnemonic"
INFURA_API_KEY="your infura project id"
```

3. Configure `truffle-config.js` with the Sepolia network (see project config).

4. Deploy:

```
truffle migrate --network sepolia --reset
```

6. Usage

After deployment, the owner can:

- Add addresses to the whitelist:
`crowdsale.addToWhitelist(address)`
- Switch between Pre-ICO and ICO stages:
`crowdsale.setCrowdsaleStage(1)` (for ICO)
- Finalize the crowdsale: `crowdsale.finalize()` – this will either enable refunds or distribute bonus tokens and unpause the token.

Investors can purchase tokens by sending ETH to the crowdsale address or calling `buyTokens(address beneficiary)` .

7. Security Considerations

- All critical functions are protected by `onlyOwner` (inherited from OpenZeppelin's `Ownable`).
- Investor caps prevent large individual purchases and ensure minimum participation.
- Refund mechanism guarantees investor funds are safe if the goal isn't met.
- Token transfers are blocked during crowdsale to prevent pre-mature trading.
- Time-locked vesting for team tokens ensures long-term commitment.
- The code is based on audited OpenZeppelin contracts, but a professional audit is recommended before mainnet deployment.

8. License

This project is released under the MIT License.
